Financial RAG Research Assistant

Implementation report | 20 October 2026

Executive summary

This capstone implements a local, evidence-first financial document question-answering pipeline. PyMuPDF extracts page text; conservative normalization preserves financial notation; word chunks retain document, filename, page, and chunk metadata; a sentence-transformer creates normalized vectors; FAISS inner-product search performs cosine-style semantic retrieval; and an extractive transformer answers only from retrieved evidence with citations.

Architecture

The reusable modules are src/preprocessing.py, src/model_utils.py, src/retrieval.py, src/inference.py, and src/evaluation.py. app.py and api.py call the same pipeline rather than duplicating business logic. Retrieval indexes are persisted as index.faiss and chunks.json under models/saved_model when explicitly saved. Notebook examples call the modules directly.

Preprocessing

Extraction is page-wise with one-based numbering and document identity. Missing, non-PDF, encrypted, empty, and unreadable files raise explicit errors. Cleaning normalizes Unicode, whitespace, common ligatures, and conservative line-break hyphenation. Chunking validates positive size and strictly smaller overlap, slices words rather than characters, retains empty-page numbering, and records exact word and character counts.

Retrieval and answering

The default embedding model is sentence-transformers/all-MiniLM-L6-v2. Encodings are float32, finite, and normalized. IndexFlatIP therefore implements cosine similarity. Search results carry the original chunk record and score. The QA model is distilbert-base-uncased-distilled-squad; answer logits are restricted to context tokens, and responses include answer text, evidence, page, chunk, document, and score. Empty or weak evidence produces an abstention.

Measured verification

A generated one-page PDF containing a financial value was processed successfully. The test produced one page and one chunk, built a 384-dimensional embedding, searched FAISS, and returned the expected page and chunk. The index was saved, reloaded, and queried again with matching source text. Preprocessing assertions covered ligatures, line-wrap repair, whitespace, exact overlap, counts, page metadata, empty pages, and invalid parameters. All four notebooks are valid nbformat 4 JSON and their code cells compile. Python compileall passed for source, app, and API.

Evaluation and limitations

No genuine financial filing or labeled question set was present in the repository, so no benchmark score is claimed. data/sample_data.csv intentionally contains unanswerable placeholders rather than fabricated facts. Evaluation helpers support Recall@k, Precision@k, MRR, exact match, grounding, abstention, and latency once real cases are supplied. OCR is not bundled; scanned PDFs fail clearly and require an external OCR pipeline. Tables and multi-column extraction remain dependent on PyMuPDF ordering.

Deployment notes

Install requirements in a Python environment with compatible binary wheels. The current validation environment is Python 3.14 and already contained the core ML stack; optional FastAPI, Streamlit, and ReportLab packages were unavailable during validation because pip resolver backtracking selected incompatible source-build paths. The declared requirements use lower bounds to avoid forcing one unavailable wheel. First model use downloads Hugging Face weights; credentials and private documents must not be committed.

Future improvements

Add a real financial-filing corpus and manually reviewed evidence labels; add OCR behind an explicit optional extra; improve table-aware extraction; add a persistent multi-document service lifecycle; calibrate abstention thresholds on held-out data; and add automated citation entailment checks.